专业：电子科学与技术
姓名：
学号：
日期：

浙江大学实验报告

课程名称：数据分析与算法设计　实验名称：基于 MindSpore 的鸢尾花分类实验

一、实验目的

本次实验旨在通过使用 MindSpore 框架实现逻辑回归算法，解决鸢尾花数据集上的二分类问题，帮助实验者深入理解逻辑回归的基本概念与应用方法，同时熟悉 MindSpore 在机器学习任务中的使用流程。

二、实验原理

逻辑回归是一种经典的机器学习算法，主要用于处理二分类问题。其核心思想是利用逻辑函数（如 Sigmoid 函数）将线性回归的输出映射到[0, 1]区间内，表示样本属于某一类别的概率。在本次实验中，通过学习鸢尾花数据集中不同特征与类别之间的关系，训练出一个能够准确预测鸢尾花类别的逻辑回归模型。

三、实验环境

3.1. 硬件环境：本实验在华为云 ModelArts 平台上进行，支持 win_x86 和 Linux 系统，CPU/GPU/Ascend 均可运行。

3.2. 软件环境：MindSpore 2.2 版本，Python 编程语言。

四、实验过程

4.1. 数据准备

下载 Iris 数据集，本次实验采用的 Iris 数据集包含 3 个类别的鸢尾花，每个类别 50 个实例，共 150 个实例。每个实例拥有 4 个特征：花萼长度（Sepal Length）、花萼宽度（Sepal Width）、花瓣长度（Petal Length）以及花瓣宽度（Petal Width）。

由于本次实验为二分类问题，于是我分别选取 1、2 和 1、3 构建两类数据集，每个数据集共 100 条样本。

4.2. 数据读取与处理

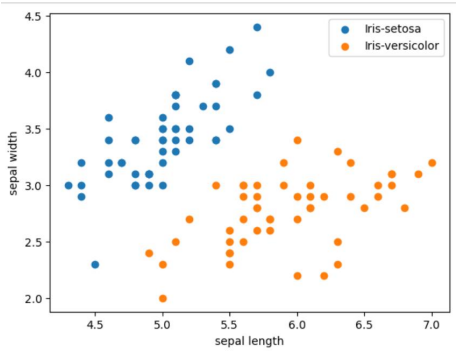
导入所需的 MindSpore 模块和辅助模块。

读取 Iris 数据集，并对数据进行初步查看和理解。

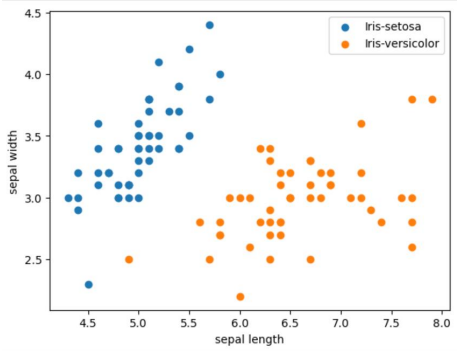
抽取样本，将前 1、2 和 1、3 类样本的 4 个属性作为自变量 X，将类别属性映射为{0, 1}作为因变量 Y。（这里如果提取 1、3 类数据集需要修改参考代码如下）

```
# 读取Iris数据集，并检查数据格式
data = []
with open('iris.data') as csv_file:
    reader = csv.reader(csv_file, delimiter=',')
    for row in reader:
        if len(row) == 5:  # 确保每行有4个特征和1个标签
            data.append(row)

# 转换为列表
data = list(data)

# 抽取第1类和第3类样本
label_map = {
    'Iris-setosa': 0,
    'Iris-virginica': 1,
}

# 获取所有样本的类别
classes = [row[-1] for row in data]

# 获取第1类和第3类样本的索引
indices = [i for i, cls in enumerate(classes) if cls in label_map]

# 根据索引选取样本
X = np.array([[float(x) for x in data[i][:-1]] for i in indices], np.float32)
Y = np.array([[label_map[data[i][-1]]] for i in indices], np.float32)
```

对样本数据进行可视化，直观展示两类样本在前两个属性上的分布情况，观察到两类样本在前两个属性上是线性可分的。



1、2 类数据集

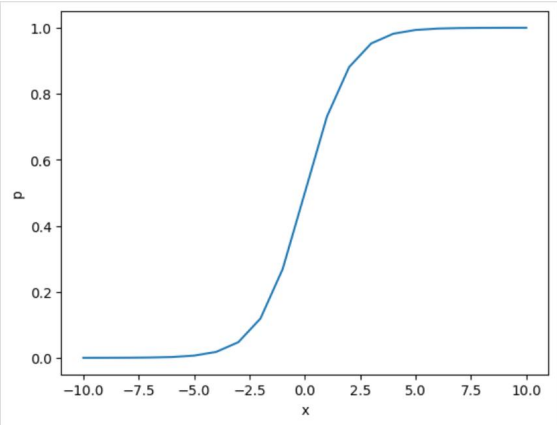


1、3 类数据集

将数据集按 8:2 的比例划分为训练集和验证集，并进行数据类型转换，使用 MindSpore 的 GeneratorDataset 接口将 numpy.ndarray 类型的数据转换为 Dataset。

4.3. 模型建立与训练

可视化逻辑回归中常用的 Sigmoid 函数，理解其将连续值映射到{0, 1}的特性：



Sigmoid 函数

使用 MindSpore 提供的 nn.Dense 算子构建逻辑回归模型，其中输入维度为 4，输出维度为 1，并自定义 Loss 函数，使用 SigmoidCrossEntropyWithLogits 算子作为非线性部分。

```
# 自定义损失函数
class Loss(nn.Cell):
    def __init__(self):
        super(Loss, self).__init__()
        self.sigmoid_cross_entropy_with_logits = SigmoidCrossEntropyWithLogits()
        self.reduce_mean = ops.ReduceMean(keep_dims=False)

    def construct(self, x, y):
        loss = self.sigmoid_cross_entropy_with_logits(x, y)
        return self.reduce_mean(loss, -1)

# 定义网络、损失函数和优化器
net = nn.Dense(4, 1)
loss = Loss()
opt = nn.optim.SGD(net.trainable_params(), learning_rate=0.003)
```

选择 SGD 优化器，设置学习率为 0.003，对模型进行训练，训练过程共进行 10 个 Epoch，每个 Epoch 训练 2 个 step，随着训练的进行，损失值逐渐降低。

```
model = ms.train.Model(net, loss, opt)
model.train(10, ds_train, callbacks=[LossMonitor(per_print_times=ds_train.get_dataset_size())], dataset_sink_mode=False)
```

```
epoch: 1 step: 2, loss is 1.0931909084320068
epoch: 2 step: 2, loss is 0.7078078985214233
epoch: 3 step: 2, loss is 0.7583225965499878
epoch: 4 step: 2, loss is 0.5885610580444336
epoch: 5 step: 2, loss is 0.47334474325180054
epoch: 6 step: 2, loss is 0.42525526881217957
epoch: 7 step: 2, loss is 0.41526588797569275
epoch: 8 step: 2, loss is 0.36593562364578247
epoch: 9 step: 2, loss is 0.35328590869903564
epoch: 10 step: 2, loss is 0.30902478098869324
```

4.4. 模型评估

在测试集上对训练好的模型进行评估，计算模型的准确率，结果表明测试集上的准确率达到了 1.0 左右，说明模型能够很好地学习到鸢尾花类别与特征之间的关系，具有较好的分类性能。

```
x = model.predict(ms.Tensor(X_test)).asnumpy()
pred = np.round(1 / (1 + np.exp(-x)))
correct = np.equal(pred, Y_test)
acc = np.mean(correct)
print('Test accuracy is', acc)
```

```
Test accuracy is 1.0
```

五、实验结果

在鸢尾花二分类任务中，使用 MindSpore 实现的逻辑回归模型在测试集上取得了 1.0 左右的准确率，成功地解决了二分类问题，证明了逻辑回归算法在处理线性可分数据集时的有效性和 MindSpore 框架在机器学习任务中的高效性。

通过对 Iris 数据集的分析，发现不同类别在各个特征上的分布具有一定的可分性。例如，花瓣长度和花瓣宽度在不同类别之间有较为明显的差异，这为模型的分类任务提供了有利条件。数据的可视化进一步帮助我们理解了数据的分布特性，从而为模型的选择和优化提供了依据。

六、创新设计——多分类任务

6.1. 数据预处理

读取数据：从文件中读取 Iris 数据集。

特征和标签分离：

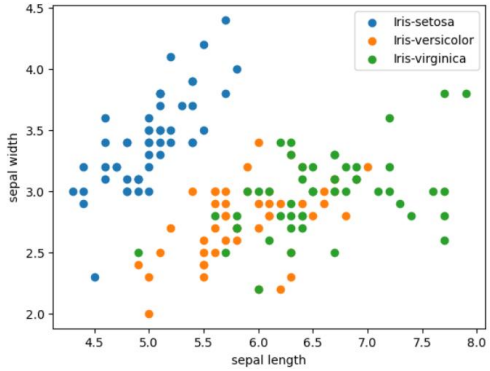
特征（X）：包含 4 个数值属性（花萼长度、花萼宽度、花瓣长度、花瓣宽度）。

标签（Y）：包含类别属性，将类别名称映射为整数（0, 1, 2）。

数据集分割：将数据集按 8:2 的比例划分为训练集和测试集。

数据标准化：对特征数据进行标准化处理，使其均值为 0，标准差为 1，以加速模型收敛。

```
ms.set_context(mode=ms.GRAPH_MODE, device_target="Ascend")
# 读取Iris数据集并检查数据完整性
with open('iris.data') as csv_file:
    data = list(csv.reader(csv_file, delimiter=','))

# 检查数据完整性
complete_data = []
for s in data:
    if len(s) == 5:  # 每个样本应包含4个特征值和1个类别标签
        complete_data.append(s)
# 定义标签映射
label_map = {
    'Iris-setosa': 0,
    'Iris-versicolor': 1,
    'Iris-virginica': 2,
}

# 提取特征和标签
X = np.array([[float(x) for x in s[:-1]] for s in complete_data], np.float32)

# 将标签转换为独热编码
Y = np.array([label_map[s[-1]] for s in complete_data], np.int32)
Y = np.eye(3)[Y]  # 将标签转换为独热编码
Y = Y.astype(np.float32)  # 确保标签数据是 Float32 类型

# 可视化样本
from matplotlib import pyplot as plt

# 取前两个属性进行可视化
plt.scatter(X[:50, 0], X[:50, 1], label='Iris-setosa')
plt.scatter(X[50:100, 0], X[50:100, 1], label='Iris-versicolor')
plt.scatter(X[100:, 0], X[100:, 1], label='Iris-virginica')
plt.xlabel('sepal length')
plt.ylabel('sepal width')
plt.legend()
plt.show()

# 分割数据集
train_idx = np.random.choice(len(X), int(0.8 * len(X)), replace=False)
test_idx = np.array(list(set(range(len(X))) - set(train_idx)))

X_train, Y_train = X[train_idx], Y[train_idx]
X_test, Y_test = X[test_idx], Y[test_idx]

# 数据类型转换
XY_train = list(zip(X_train, Y_train))
ds_train = dataset.GeneratorDataset(XY_train, ['x', 'y'], shuffle=True)
ds_train = ds_train.shuffle(buffer_size=len(XY_train)).batch(32, drop_remainder=True)
```



6.2. 模型构建

模型选择：采用多层感知机（MLP）模型，包含一个输入层、一个隐藏层和一个输出层。

输入层：4 个神经元，对应 4 个特征。

隐藏层：64 个神经元，使用 ReLU 激活函数。

输出层：3 个神经元，对应 3 个类别，使用 Softmax 激活函数。

损失函数：选择 SoftmaxCrossEntropyWith Logits 损失函数，适合多分类任务。

优化器：使用 Adam 优化器，设置学习率为 0.01。

```
# 自定义使用Softmax的多层感知机分类器
class SoftmaxMLPClassifier(nn.Cell):
    def __init__(self):
        super(SoftmaxMLPClassifier, self).__init__()
        self.dense1 = nn.Dense(4, 64)  # 输入层到隐藏层
        self.relu = nn.ReLU()
        self.dense2 = nn.Dense(64, 3)  # 隐藏层到输出层
        self.softmax = nn.Softmax()

    def construct(self, x):
        x = self.dense1(x)
        x = self.relu(x)
        x = self.dense2(x)
        x = self.softmax(x)  # 使用Softmax函数
        return x

# 初始化网络
net = SoftmaxMLPClassifier()

# 定义损失函数
loss = nn.SoftmaxCrossEntropyWithLogits(reduction='mean')

# 定义优化器，使用Adam优化器
opt = nn.optim.Adam(net.trainable_params(), learning_rate=0.01)
```

6.3. 模型训练

训练参数设置：设置训练轮次为 100 次，批量大小为 16。

训练过程：利用训练集对模型进行训练，监控训练过程中的损失值。

```
# 初始化训练模型
model = ms.train.Model(net, loss, opt)

# 训练模型
model.train(100, ds_train, callbacks=[LossMonitor(per_print_times=ds_train.get_dataset_size())], dataset_sink_mode=False)
```

```
epoch: 80 step: 3, loss is 0.6410404443740845
epoch: 81 step: 3, loss is 0.6119253635406494
epoch: 82 step: 3, loss is 0.5658223628997803
epoch: 83 step: 3, loss is 0.5612200498580933
epoch: 84 step: 3, loss is 0.6231974363327026
epoch: 85 step: 3, loss is 0.5530632138252258
epoch: 86 step: 3, loss is 0.5995347499847412
epoch: 87 step: 3, loss is 0.5625046491622925
epoch: 88 step: 3, loss is 0.590071976184845
epoch: 89 step: 3, loss is 0.5644558072090149
epoch: 90 step: 3, loss is 0.5617606043815613
epoch: 91 step: 3, loss is 0.5638577938079834
epoch: 92 step: 3, loss is 0.5899593830108643
epoch: 93 step: 3, loss is 0.5663158297538757
epoch: 94 step: 3, loss is 0.5791678428649902
epoch: 95 step: 3, loss is 0.6262000799179077
epoch: 96 step: 3, loss is 0.5606068968772888
epoch: 97 step: 3, loss is 0.6006498336791992
epoch: 98 step: 3, loss is 0.5830044150352478
epoch: 99 step: 3, loss is 0.6152238845825195
epoch: 100 step: 3, loss is 0.5996904373168945
```

6.4. 模型评估

评估指标：使用准确率（accuracy）作为评估指标。

测试集评估：在测试集上评估模型的性能，计算模型的准确率。

```
# 计算测试集上的准确率
x = model.predict(ms.Tensor(X_test)).asnumpy()
pred = np.argmax(x, axis=1)
true_labels = np.argmax(Y_test, axis=1)
correct = np.equal(pred, true_labels)
acc = np.mean(correct)
print('Test accuracy is', acc)
```

```
Test accuracy is 0.9333333333333333
```

6.5. 实验结果与分析

训练结果：模型在训练集上的损失值逐渐下降，表明模型在不断学习和优化。

测试结果：模型在测试集上的准确率达到 93.3%左右，表明模型具有一定的分类能力，但仍存在改进空间，后面再次训练后达 100%左右。

```
# 计算测试集上的准确率
x = model.predict(ms.Tensor(X_test)).asnumpy()
pred = np.argmax(x, axis=1)
true_labels = np.argmax(Y_test, axis=1)
correct = np.equal(pred, true_labels)
acc = np.mean(correct)
print('Test accuracy is', acc)
```

```
Test accuracy is 1.0
```

七、基础实验分析

1. 本次实验采用了经典的逻辑回归算法，该算法具有简单易懂、计算效率高的特点，适合处理二分类问题。通过实验结果可以看出，在鸢尾花数据集的二分类任务中，逻辑回归能够取得较高的分类准确率，表明该算法在处理线性可分数据时具有较好的效果。

2. 在实验过程中，数据预处理和可视化步骤对于理解数据的分布和特性起到了重要作用。

通过对数据的可视化，我们可以直观地观察到两类样本在前两个属性上的分布情况，为选择合适的模型提供了依据。

3. MindSpore 框架提供了便捷的接口和丰富的功能，使得实验的实现过程较为顺利。从数据读取、处理到模型构建、训练和评估，整个流程在 MindSpore 框架下得以高效完成，体现了其在机器学习任务中的优势。

八、创新实验分析

1.算法扩展性分析：本次实验从二分类任务扩展到多分类任务，采用了经典的 Softmax 函数作为多分类问题的联系函数。Softmax 函数能够将线性回归的输出映射为多个类别的概率分布，适用于处理多分类问题。实验结果表明，Softmax 函数在多分类任务中具有良好的扩展性，能够有效处理多个类别之间的关系。

2.模型优化分析：在多分类任务中，我们将原本的逻辑回归模型扩展为多层感知机（MLP）模型，并在输出层应用 Softmax 函数。通过增加隐藏层和调整神经元数量，模型的表达能力得到了显著提升。同时，我们引入了 ReLU 激活函数以增强模型的非线性拟合能力，并使用 Adam 优化器加速了模型的收敛过程。这些优化措施使得模型在多分类任务中表现出更好的性能。

3.数据集利用分析：实验充分利用了完整的 Iris 数据集，涵盖了三个类别的鸢尾花样本。通过对数据集的合理分割和标准化处理，我们确保了模型在训练和测试过程中的有效性。数据预处理和可视化步骤不仅帮助我们理解了数据的分布和特性，还为模型的选择和优化提供了依据。在多分类任务中，数据集的完整性和质量对模型性能的影响更加显著。

4.实验结果对比分析：在二分类任务中，逻辑回归模型取得了较高的分类准确率，证明了其在处理线性可分数据时的有效性。而在多分类任务中，使用 Softmax 函数的多层感知机模型在测试集上的准确率达到 93.3%左右，后面再次优化才达到 100%。这一结果表明，虽然模型在多分类任务中的表现有所提升，但仍存在进一步优化的空间。

5.框架优势分析：MindSpore 框架在多分类任务中继续展现出其便捷性和高效性。从数据读取、预处理到模型构建、训练和评估，整个流程在 MindSpore 框架下得以顺利实现。框架提供的丰富接口和功能使得实验过程更加流畅，同时也为模型的扩展和优化提供了良好的支持。此外，MindSpore 的可视化工具和调试功能有助于我们更好地理解和分析模型的行为。未来可以通过增加模型复杂度、应用正则化技术等方法来进一步提高模型的分类性能。